

Lab 2 report

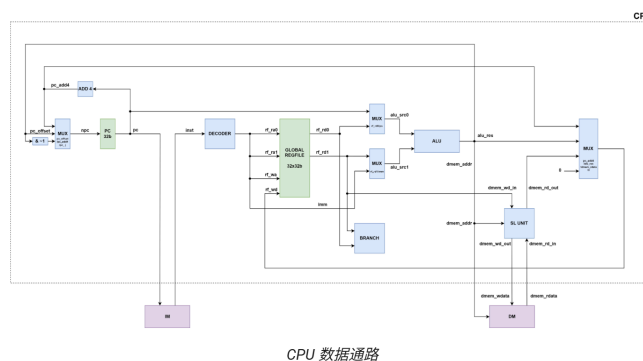
PB24111630 黄雯佩

实验目的与内容

1. 更加熟悉RV32I跳转、访存等指令
2. 明白RV32I具体的代码实现
3. 懂得CPU的整体框架

逻辑设计

1. 请参照PPT画出你设计的各模块的框图和数据通路



按照实验文档设计

2. 如果存在状态机，请绘制出状态机的状态转换图；
3. 请贴出较为核心的代码设计代码，并加以解释说明。

```

24 // --- 访存处理逻辑 (Load) ---
25 always @(*) begin
26     case (dmem_access)
27     LB: begin // 加载字节并符号扩展
28         case (offset)
29             2'b00: rd_out = {{24{rd_in[7]}}, rd_in[7:0]};
30             2'b01: rd_out = {{24{rd_in[15]}}, rd_in[15:0]};
31             2'b10: rd_out = {{24{rd_in[23]}}, rd_in[23:0]};
32             2'b11: rd_out = {{24{rd_in[31]}}, rd_in[31:0]};
33         endcase
34     end
35     LBU: begin // 加载字节并零扩展
36         case (offset)
37             2'b00: rd_out = {24'b0, rd_in[7:0]};
38             2'b01: rd_out = {24'b0, rd_in[15:0]};
39             2'b10: rd_out = {24'b0, rd_in[23:0]};
40             2'b11: rd_out = {24'b0, rd_in[31:0]};
41         endcase
42     end
43     LH: begin // 加载半字并符号扩展
44         if (offset == 2'b00) rd_out = {{16{rd_in[15]}}, rd_in[15:0]};
45         else if (offset == 2'b10) rd_out = {{16{rd_in[31]}}, rd_in[31:0]};
46         else rd_out = rd_in; // 非对齐访问不做处理
47     end
48     LHU: begin // 加载半字并零扩展
49         if (offset == 2'b00) rd_out = {16'b0, rd_in[15:0]};
50         else if (offset == 2'b10) rd_out = {16'b0, rd_in[31:0]};
51         else rd_out = rd_in; // 非对齐访问不做处理
52     end
53     LW: begin // 加载数字
54         rd_out = rd_in;
55     end
56     default: rd_out = rd_in;
57 endcase
58 end

```

load逻辑，依据DECODER得出dmem_access,的有符号扩展和无符号扩展分开处理

```
31 wire signed [31:0] s_src0 = br_src0;
32 wire signed [31:0] s_src1 = br_src1;
33
34 always @(*) begin
35     // 默认不跳转, 执行顺序执行 (PC + 4)
36     npc_sel = 2'b00;
37
38     case (br_type)
39         BEQ: if (br_src0 == br_src1) npc_sel = 2'b01;
40         BNE: if (br_src0 != br_src1) npc_sel = 2'b01;
41
42         // 有符号比较
43         BLT: if (s_src0 < s_src1) npc_sel = 2'b01;
44         BGE: if (s_src0 >= s_src1) npc_sel = 2'b01;
45
46         // 无符号比较
47         BLTU: if (br_src0 < br_src1) npc_sel = 2'b01;
48         BGEU: if (br_src0 >= br_src1) npc_sel = 2'b01;
49
50         // 无条件跳转 (J-type)
51         JAL: npc_sel = 2'b01; // 跳转到 PC + imm
52         JALR: npc_sel = 2'b10; // 跳转到 rs1 + imm (通常 npc_sel=2 对应单独的路径)
53
54         default: npc_sel = 2'b00;
55     endcase
56 end
57
```

branch逻辑

仿真结果与分析

1. 请给出仿真文件的运行结果截图，并对结果加以阐释；

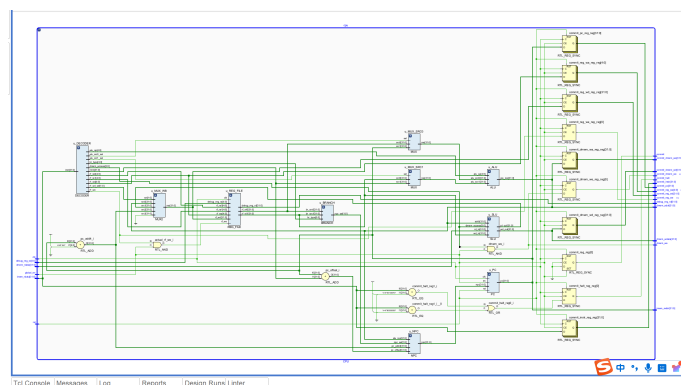
```
31 wire signed [31:0] s_src0 = br_src0;
32 wire signed [31:0] s_src1 = br_src1;
33
34 always @(*) begin
35     // 默认不跳转, 执行顺序执行 (PC + 4)
36     npc_sel = 2'b00;
37
38     case (br_type)
39         BEQ: if (br_src0 == br_src1) npc_sel = 2'b01;
40         BNE: if (br_src0 != br_src1) npc_sel = 2'b01;
41
42         // 有符号比较
43         BLT: if (s_src0 < s_src1) npc_sel = 2'b01;
44         BGE: if (s_src0 >= s_src1) npc_sel = 2'b01;
45
46         // 无符号比较
47         BLTU: if (br_src0 < br_src1) npc_sel = 2'b01;
48         BGEU: if (br_src0 >= br_src1) npc_sel = 2'b01;
49
50         // 无条件跳转 (J-type)
51         JAL: npc_sel = 2'b01; // 跳转到 PC + imm
52         JALR: npc_sel = 2'b10; // 跳转到 rs1 + imm (通常 npc_sel=2 对应单独的路径)
53
54         default: npc_sel = 2'b00;
55     endcase
56 end
57
```

和正确结果一致

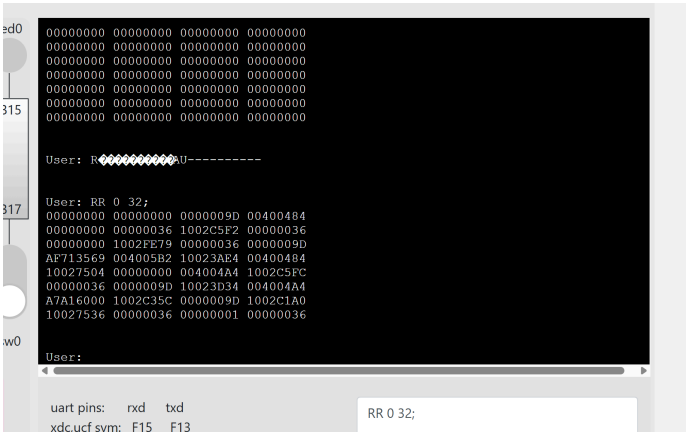
2. 请贴出有特点的仿真测试文件，并说明你在编写仿真测试文件时，对各类情况的考虑（选做）。
依据实验文档仿真

电路设计与分析

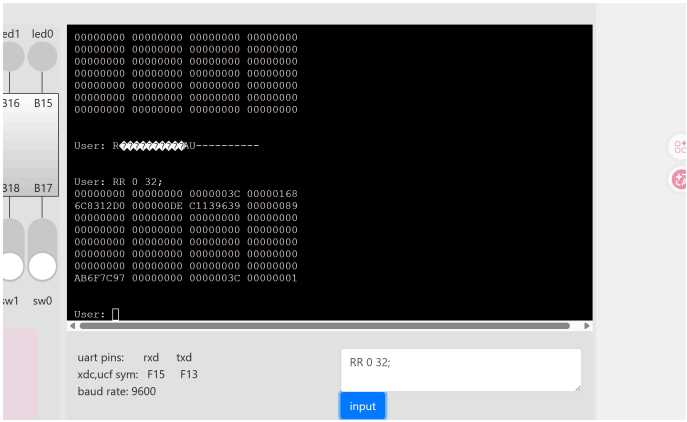
1. 请给出完整的RTL电路图。若某模块较为复杂，也可以再给出该模块的RTL电路图；



测试结果与分析



PDU仿真结果



斐波那契大数数列，n取60

任务4

1. 掩码访问

(1)接口定义

输入：

addr[1:0]：访存地址的低 2 位。

cpu_wdata：CPU 准备写入的 32 位原始数据。

mem_rdata：从存储器 spo 端口读出的 32 位数据。

MemOp：访存控制信号（指示 lw, lh, sb 等具体指令）。

输出：

mask：4 位掩码信号，输出给存储器。

mem_wdata：处理后准备输出给存储器 d 端口的 32 位写数据。

cpu_rdata：处理后准备写回 CPU 寄存器的 32 位读数据。

(2)思路：

在原有的设计中，SLT需要承担复杂的“读-改-写”逻辑来处理非字对齐或部分字节的写入。一旦DMEM支持mask接口，SLT大幅简化，核心任务转变为：根据指令类型MemOp和访存地址低两位生成对应的4位mask，并在写操作时对准数据轨道，在读操作时还原数据并进行符号扩展。

(3)store:

Store Word(sw / st.w)：

mask = 4'b1111

```
mem_wdata = cpu_wdata
```

Store Halfword (sh / st.h):

若 $\text{addr}[1] == 0$: $\text{mask} = 4'b0011$, $\text{mem_wdata} = \text{cpu_wdata}$

若 $\text{addr}[1] == 1$: $\text{mask} = 4'b1100$, $\text{mem_wdata} = \text{cpu_wdata} \ll 16$

Store Byte (sb / st.b):

根据 $\text{addr}[1:0]$ 的值 (00, 01, 10, 11), 生成对应的独热码掩码 (4'b0001, 4'b0010, 4'b0100, 4'b1000)。

```
mem_wdata = cpu_wdata << (addr[1:0] * 8)
```

(4)load

第一步：移位还原。将存储器返回的 mem_rdata 统一进行逻辑右移： $\text{shifted_data} = \text{mem_rdata} \gg (\text{addr}[1:0] * 8)$, 将目标有效数据移至最低位。

第二步：符号扩展。根据 MemOp 判断扩展类型：

无符号加载 (lhu / lbu 等)：由于未命中字节已被存储器置 0, 且进行了逻辑右移, 此时高位自带补 0, $\text{cpu_rdata} = \text{shifted_data}$, 无需额外操作。

有符号加载 (lh / lb 等)：提取 shifted_data 的对应符号位, 将其填充至高位构成 32 位数据输出给 cpu_rdata 。

2. 可能路径

PC 输出 -> 指令存储器(IM)取指 -> 寄存器堆(RF)读寄存器数据 -> ALU 进行基址+偏移地址的加法 -> 数据存储器(DMEM)根据地址读取数据 -> SL_UNIT 处理掩码和符号扩展 -> 多路选择器(MUX)选通 -> 寄存器堆(RF)写入数据

总结

1. 请对本次实验中你完成的任务进行简要总结, 并总结自己的收获和体验;
完成了单周期CPU设计
更加熟悉verilog使用
2. 如果对本次实验的设计或助教、老师有建议, 可以在这里写下, 助教和老师会认真阅读并讨论.